Practice 09 — Containerize an App and Back It Up to S3

Objective: run a container, write a Dockerfile for a small Python app, see the layer cache at work, keep the app's data in a volume, then wire it up with **Docker Compose** next to an S3 service and back the data up to S3 with a one-off container. These are the skills from today's lecture.

Timebox: ~55–60 min, in class: about 20 min for Part A (Docker basics), 25 min for Part B (Compose + S3), 5–10 min for cleanup. If you're stuck on one step for more than 10 minutes, ask instead of pushing through alone.

> **How you're checked:** this practice is done in class and **checked orally** at the end. There's no written report. Keep your terminal open, be ready to **show** each item in the Definition of done and **explain** what each step does. The instructor will ask you a few of the questions under "Be ready to explain".

> **Before class (at home):** install **Docker Desktop** (Windows/macOS) or Docker Engine (Linux) and check that `docker run hello-world` and `docker compose version` work. Downloading images over the university Wi-Fi with 25 laptops at once is slow, so also pull the images once: `docker pull python:3.12-slim nginx:1.31-alpine motoserver/moto:5.2.5 amazon/aws-cli:2.39.3` (pull them one by one if your Docker version wants one image per command).

> **Two tracks, for Part B only.** Part A is the same for everyone and runs on your own laptop. In Part B, **Track B** backs up to a local S3 (moto, running as a container); **Track A** backs up to a real S3 bucket in **AWS Academy Learner Lab**. Tell the instructor which track you did.

Definition of done (things you can show)

☐ `nginx` running in a container on `localhost:8080`, its **logs**, and a command run **inside** it with `docker exec`
☐ Your **`Dockerfile`**, and the image **`lab09-app:1.0`** in `docker images`
☐ The app answering on `localhost:8000` with its container hostname and a file list
☐ A rebuild after editing `app.py` where the **`pip install`** **step is `CACHED`**, and you can say why
☐ A note created through the app that **survives `docker rm`** of the container, because `/data` is a **named volume**
☐ `docker compose ps` with **`app`** and **`s3`** running, and the `app` container resolving the name **`s3`**
☐ The **`backup`** service run twice: the files from `/data` in your bucket under `backups/<date>/<time>/`, listed with `aws s3 ls`

□ Cleanup: no lab containers, volumes or networks left (Track A: the bucket deleted and the lab ended)

Part A — Docker basics (everyone, ~20 min)

A1. Run, inspect, stop a container

```
docker run -d --name web -p 8080:80 nginx:1.31-alpine
docker ps                          # STATUS: Up …, PORTS: 0.0.0.0:8080->80/tcp
curl -s localhost:8080 | grep title   # <title>Welcome to nginx!</title>
docker logs web                    # one line per request
docker exec -it web sh             # a shell INSIDE the container
#   nginx -v ; ls /usr/share/nginx/html ; exit
docker stop web && docker rm web
```

`-d` runs it in the background, `-p 8080:80` publishes container port 80 on your laptop's port 8080, `--name` saves you from copying IDs.

A2. The app

Make a folder `lab09` and put three files in it. You don't need to change the app.

`app.py`:

```
import os, socket
from flask import Flask

app = Flask(__name__)
DATA = os.environ.get("DATA_DIR", "/data")

@app.get("/")
def index():
    os.makedirs(DATA, exist_ok=True)
    return {"host": socket.gethostname(), "files": sorted(os.listdir(DATA))}

@app.post("/notes/<name>")
def add_note(name):
    os.makedirs(DATA, exist_ok=True)
    with open(os.path.join(DATA, f"{name}.txt"), "w") as f:
        f.write(f"note {name}\n")
    return {"saved": f"{name}.txt"}, 201

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000)
```

`requirements.txt`:

```
flask==3.1.3
```

`.dockerignore` (what **not** to send into the build):

```
.venv/
__pycache__/
.env
*.tar.gz
```

A3. Write the Dockerfile, build, run

Create `Dockerfile` in the same folder. Fill in the TODOs; every instruction you need is on the lecture slides.

```
FROM python:3.12-slim
WORKDIR /app
# TODO 1: copy ONLY requirements.txt, then install it with
#         pip install --no-cache-dir -r requirements.txt
# TODO 2: copy app.py
ENV DATA_DIR=/data
EXPOSE 8000
# TODO 3: start the app with python (use the JSON "exec" form)
```

```
docker build -t lab09-app:1.0 .
docker images lab09-app
docker run -d --name app -p 8000:8000 lab09-app:1.0
curl -s localhost:8000/                       # {"files":[],"host":"<container id>"}
curl -s -X POST localhost:8000/notes/first    # {"saved":"first.txt"}
curl -s localhost:8000/                       # {"files":["first.txt"],"host":"…"}
docker logs app                               # GET / 200, POST /notes/first 201
docker rm -f app
```

The `host` in the answer is the **container's** hostname (its short ID), not your laptop's.

A4. See the layer cache

Add a comment line to the end of `app.py` (any text after `#`), then build again:

```
docker build -t lab09-app:1.0 .
```

In the output, the `pip install` step says **`CACHED`** and only the `COPY app.py` step runs again. Now look at your Dockerfile and explain why.

Compare with the "slow" order. Save this as `Dockerfile.slow`:

```
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
ENV DATA_DIR=/data
EXPOSE 8000
CMD ["python", "app.py"]
```

```
docker build -f Dockerfile.slow -t lab09-app:slow .
# change app.py again, then:
docker build -f Dockerfile.slow -t lab09-app:slow .    # pip install runs AGAIN
docker rmi lab09-app:slow
```

A5. Keep the data in a named volume

```
docker volume create lab09-data
docker run -d --name app -p 8000:8000 -v lab09-data:/data lab09-app:1.0
curl -s -X POST localhost:8000/notes/keepme
docker rm -f app                                    # the container is gone…
docker run -d --name app -p 8000:8000 -v lab09-data:/data lab09-app:1.0
curl -s localhost:8000/        # …but "keepme.txt" is still listed, with a NEW host
docker rm -f app
```

Run it once more **without** `-v`: the file list is empty, because the container's own filesystem is thrown away with the container.

Part B — Compose + S3 (~25 min)

B1. `compose.yaml`

In the same folder:

```
services:
  app:
    build: .
    image: lab09-app:1.0
    ports:
      - "8000:8000"
    volumes:
      - appdata:/data

  s3:
    image: motoserver/moto:5.2.5
    ports:
      - "5050:5000"   # host 5050: on macOS port 5000 is taken by AirPlay

  backup:
    image: amazon/aws-cli:2.39.3
    profiles: ["tools"]        # only runs when you call it
    env_file: .env
    volumes:
      - appdata:/data:ro       # same volume as the app, read-only
    entrypoint: ["/bin/sh", "-c"]
    command:
      - |
        set -e
        aws s3 mb "s3://$$BUCKET" 2>/dev/null || true
        KEY="backups/$$(date +%F)/$$(date +%H%M%S)"
        aws s3 cp /data "s3://$$BUCKET/$$KEY/" --recursive
        echo "Backed up /data to s3://$$BUCKET/$$KEY/"
        aws s3 ls "s3://$$BUCKET/backups/" --recursive

volumes:
  appdata:
```

`$$` is how you write a literal `$` in a compose file: the variable is expanded **inside** the backup container, not by Compose. `profiles` keeps `backup` out of `docker compose up`; it's a one-off job you start with `docker compose run`.

B2. Track B — back up to the local S3 (moto)

Create `.env` (it's in `.dockerignore` and must never be committed):

```
# Track B: local S3 (moto). Never commit this file.
AWS_ACCESS_KEY_ID=test
AWS_SECRET_ACCESS_KEY=test
AWS_DEFAULT_REGION=us-east-1
AWS_ENDPOINT_URL=http://s3:5000
BUCKET=lab09-backups
```

`http://s3:5000`: inside the Compose network, the **service name** `s3` is a DNS name, and 5000 is the **container** port.

```
docker compose up -d --build
docker compose ps                                # app and s3: running
docker compose exec app getent hosts s3          # 192.168.x.x   s3
curl -s -X POST localhost:8000/notes/alpha
curl -s -X POST localhost:8000/notes/beta
docker compose run --rm backup
#   make_bucket: lab09-backups
#   upload: ../data/alpha.txt to s3://lab09-backups/backups/<date>/<time>/alpha.txt
#   Backed up /data to s3://lab09-backups/backups/<date>/<time>/
curl -s -X POST localhost:8000/notes/gamma
docker compose run --rm backup                   # a second, newer folder
```

List the bucket from the `backup` image (no AWS CLI needed on your laptop):

```
docker compose run --rm --entrypoint aws backup s3 ls s3://lab09-backups/backups/ --recursive
```

If you have the AWS CLI installed, the same works from your laptop through the published port:

```
AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test AWS_DEFAULT_REGION=us-east-1 \
  aws --endpoint-url http://localhost:5050 s3 ls s3://lab09-backups/backups/ --recursive
```

The date in the key is the container's clock, which is **UTC**.

B3. Track A — back up to real S3 (Learner Lab)

> **Using AWS Academy Learner Lab**
>
> 1. **Modules** → **Launch AWS Academy Learner Lab** → **Start Lab**, wait for the **green** dot.
> 2. Region **`us-east-1`**. Per the Learner Lab Readme, only `us-east-1` and `us-west-2` are allowed. S3 is expected to be available; if a command returns `AccessDenied`, do Track B instead.
> 3. **AWS Details** → **AWS CLI** → **Show**: copy the three values (`aws_access_key_id`, `aws_secret_access_key`, `aws_session_token`). They are **temporary**; copy them again each session.

> 4. When you're done: delete the bucket (Cleanup), then **End Lab**.

Replace `.env` with your Learner Lab credentials, and **no** `AWS_ENDPOINT_URL` line, so the CLI talks to real AWS:

```
# Track A: real S3 in Learner Lab. Never commit this file.
AWS_ACCESS_KEY_ID=<from AWS Details>
AWS_SECRET_ACCESS_KEY=<from AWS Details>
AWS_SESSION_TOKEN=<from AWS Details>
AWS_DEFAULT_REGION=us-east-1
BUCKET=lab09-<your-student-id>
```

Then do exactly the same as Track B from `docker compose up -d --build` onwards (you can ignore the `s3` service; it isn't used). Check the bucket in the S3 console too: the `backups/<date>/<time>/` folders are there.

The keys live only in `.env`, are passed in when the container starts, and are **never** inside the image: run `docker history lab09-app:1.0` or look at the Dockerfile — there's nothing secret in it.

Stretch (optional)

- **Tag for a registry:** `docker tag lab09-app:1.0 <dockerhub-user>/lab09-app:1.0`, then `docker login` and `docker push …` if you have a Docker Hub account. Explain what `docker pull` on another machine would get.
- **Nightly backups:** write the cron line that would run `docker compose run --rm backup` every night at 02:30.

Cleanup

```
docker compose down -v          # containers, network AND the appdata volume
docker rm -f app web 2>/dev/null
docker volume rm lab09-data
docker rmi lab09-app:1.0
docker ps -a ; docker volume ls # nothing from lab09 left
```

Track A, also:

```
export AWS_ACCESS_KEY_ID=… AWS_SECRET_ACCESS_KEY=… AWS_SESSION_TOKEN=… AWS_DEFAULT_REGION=us-east-
1
aws s3 rb "s3://lab09-<your-student-id>" --force
```

(or empty and delete it in the console), then **End Lab**.

Be ready to explain

1. What's the difference between an **image** and a **container**? What did `docker rm` delete, and what did it not?
2. Why was `pip install` `CACHED` after you edited `app.py`, but not with `Dockerfile.slow`?
3. Your note survived `docker rm`. Where was it stored? What's the difference between a **named volume** and a **bind mount**?
4. In `.env`, why is the S3 address `http://s3:5000` and not `localhost:5050`?
5. `CMD ["python", "app.py"]`: what's the difference between `CMD` and `ENTRYPOINT`, and why did the `backup` service override the entrypoint?
6. Why must the AWS keys be in `.env` and not in the Dockerfile or the image? What would `docker push` publish otherwise?
7. Track A vs Track B used the **same** compose file and image. What exactly changed, and why is that useful?
8. `docker compose down` vs `docker compose down -v`: what's the difference for your data?

Common mistakes

- **`port is already allocated`.** Something else (an old container) uses 8000 or 8080. `docker ps -a`, then `docker rm -f <name>`.
- **`Conflict. The container name "/app" is already in use`.** You didn't remove the previous container. `docker rm -f app`.
- **`curl: (52) Empty reply` / connection refused on 8000.** The app listens on `127.0.0.1` inside the container, or the container exited. Check `docker logs app`; the app must bind `0.0.0.0`.
- **`pip install` runs on every build.** You copied the whole folder before installing requirements. Copy `requirements.txt` first.
- **The file list is empty after recreating the container.** You forgot `-v lab09-data:/data`, or typed a different volume name (a new, empty volume is created silently).
- **`Could not connect to the endpoint URL: "http://s3:5000/…"`.** The `s3` service isn't running (`docker compose up -d`), or you ran the CLI on your laptop, where `s3` means nothing: use `http://localhost:5050` there.
- **403 `Forbidden` from `localhost:5000` on a Mac.** Port 5000 belongs to AirPlay Receiver. That's why moto is published on **5050**.
- **Track A: `ExpiredToken` / `InvalidClientTokenId`.** The Learner Lab session ended, or `AWS_SESSION_TOKEN` is missing from `.env`. Start Lab again and copy fresh values.
- **Track A: the backup went to moto (or Track B went to AWS).** Check `.env`: the `AWS_ENDPOINT_URL` line decides where every call goes.
- **`.env` ended up in Git.** Add it to `.gitignore`, and if it held real keys, treat them as leaked.